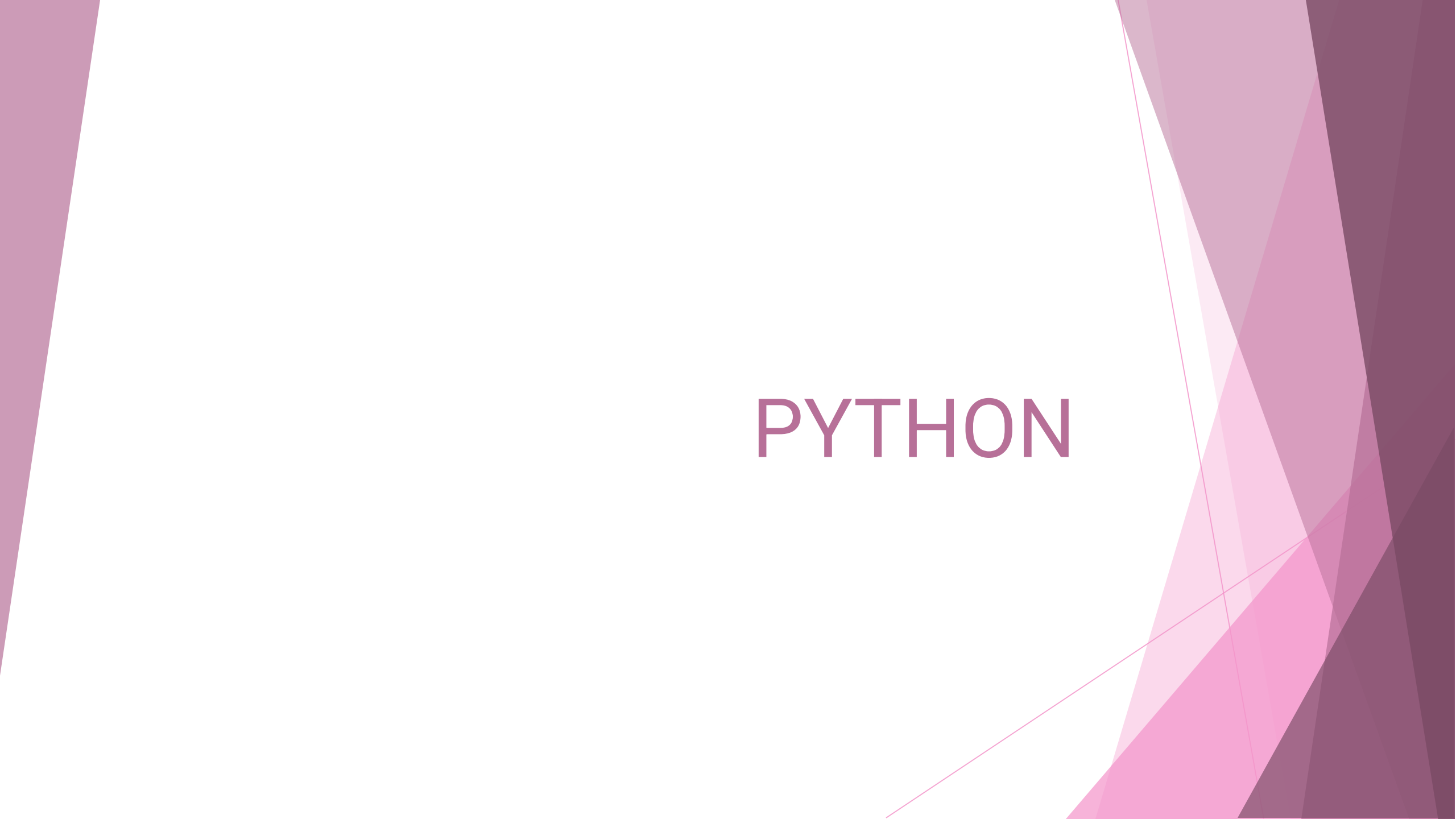




PYTHON

Python Variables, Constants and Literals

A variable is a named location used to store data in the memory.

Example: 1 :Declaring and assigning value to a variable

```
website = "apple.com"
```

```
print(website)
```

output: apple.com

Example 2: Changing the value of a variable

```
website = "apple.com"
```

```
print(website)
```

assigning a new value to website

```
website = "programiz.com"
```

```
print(website)
```

Output

Multiple Assignment:

- Multiple assignment can be done in Python at a time.

Assigning single value to multiple variable

```
x=y=z=50
```

```
print (x)
```

```
print (y)
```

```
print (z)
```

```
===== RESTART: D:/6.12.17 python/message.py =====  
50  
50  
50  
>>>
```

Multiple Assignment:

2. Assigning multiple values to multiple variables:

```
a,b,c=5,10,15
```

```
print (a)
```

```
print (b)
```

```
print (c)
```

```
===== RESTART: D:/6.12.17 python/message.py =====
```

```
5
```

```
10
```

```
15
```

```
>>>
```

Assigning multiple values to multiple variables

```
a, b, c = 5, 3.2, "Hello"
```

```
print (a)
```

```
print (b)
```

```
print (c)
```

```
x = y = z = "same"
```

```
print (x)
```

```
print (y)
```

```
print (z)
```

String literals

- A string literal is a sequence of characters surrounded by quotes. We can use both **single, double, or triple** quotes for a string.

Example

```
x = "This is Python"
```

```
y = 'C'
```

```
z = """This is a multiline string with more than one line code."""
```

```
print(x)
```

```
print(y)
```

```
print(z)
```

```
=== RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/x.py ===  
This is Python  
C  
This is a multiline string with more than one line code.  
>>> |
```

Python Comments

- In Python, we use the hash (#) symbol to start writing a comment.

Single line comment

```
#This is a comment
```

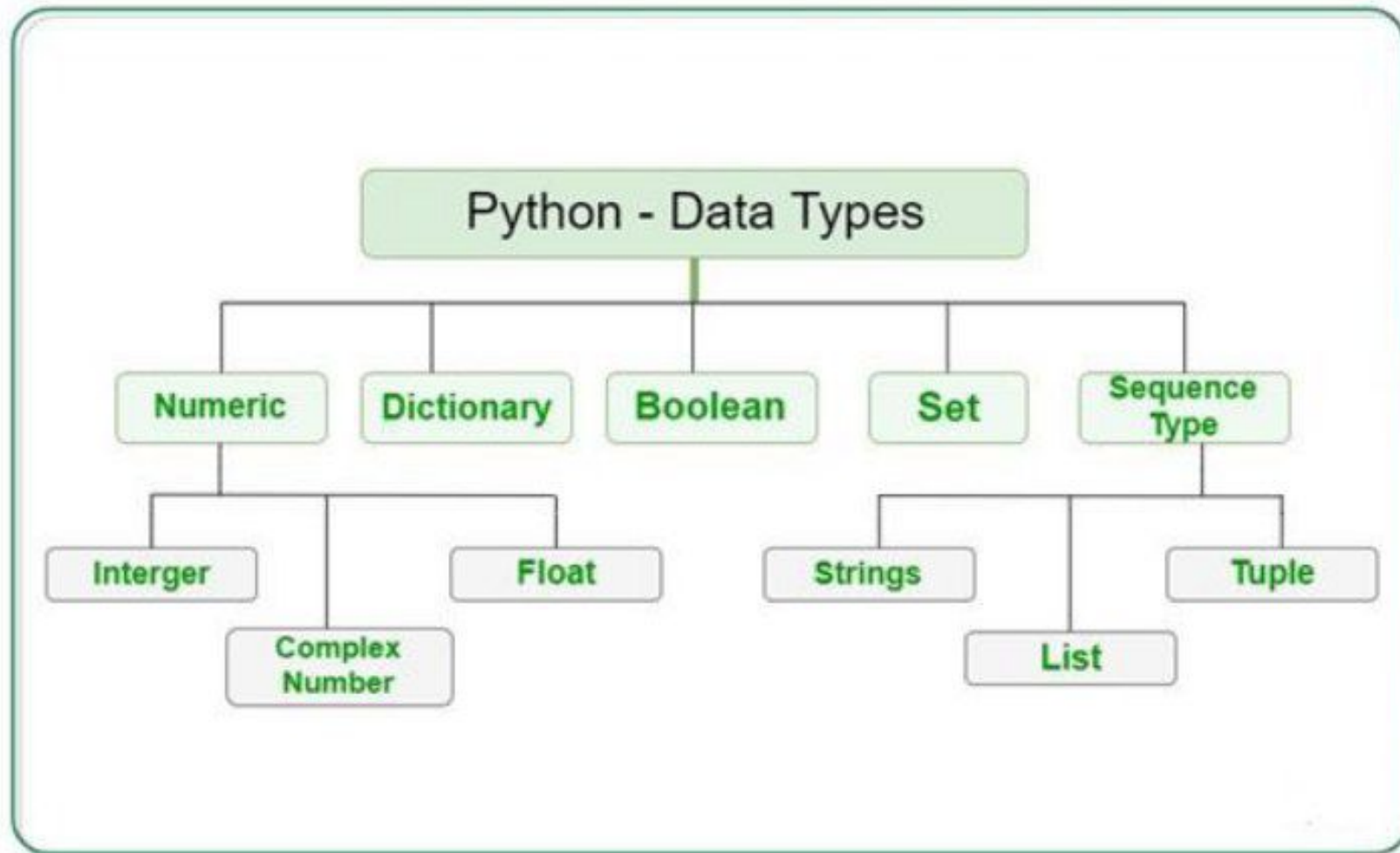
```
#print out
```

```
Hello print('Hello')
```

Multiline comment

```
"""This is also a perfect example of multi-line comments"""
```

Python Data Type:



Datatypes

Python has five standard data types :-

Numeric (Integer,Float)

String (strings)

Boolean

List

Tuple

Dictionary

```
# Python program to
# demonstrate numeric value
a = 5
print("Type of a: ", type(a))
b = 5.0
print("\nType of b: ", type(b))
c="welcome"
Print("\n Type of c:",type(c))
```

```
=== RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37
Type of a: <class 'int'>
Type of b: <class 'float'>
Type of c: <class 'str'>
```

Python Strings

```
ch = 'Hello Python'  
str1 = "String Chapter"  
print ("First value is: ", ch)  
print ("Second value is: ", str1)
```

```
>>> ===== RESTART =====  
>>>  
First value is:  Hello Python  
Second value is:  String Chapter  
>>>
```

Creating a String

with double Quotes

```
print("\nString with the use of Double Quotes: ")  
String1 = "I'm a Elysian"  
print(String1)  
print(type(String1))
```

Creating a String

with triple Quotes

```
String1 = "I'm a Elysianand I live in a world of "Elysium"  
print("\nString with the use of Triple Quotes: ")  
print(String1)  
print(type(String1))
```

RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python

String with the use of Double Quotes:

I'm a Elysian

<class 'str'>

String with the use of Triple Quotes:

I'm a Elysianand I live in a world of "Elysium"

<class 'str'>

Python Strings Using Slice

```
ch = "Hello Python"
str1 = "String Chapter"
print ("First single sub-string is: ", ch[0])
print ("Set of sub-string is: ", str1[2:6])
```

```
>>> ===== RESTART =====
>>>
First single sub-string is:  H
Set of sub-string is:  ring
>>> |
```

Python Strings using Slice

```
str = 'Hello World!'
```

```
print (str )           # Prints complete string
```

```
print (str[0])         # Prints first character of the string
```

```
print (str[2:5])       # Prints characters starting from 3rd to 5th
```

```
print (str[2:])        # Prints string starting from 3rd character
```

```
print (str * 2)        # Prints string two times
```

```
print (str + "TEST")   # Prints concatenated string
```

Python Strings

```
===== RESTART: D:/6.12.17 python/message.py =====  
Hello World!  
H  
llo  
llo World!  
Hello World!Hello World!  
Hello World!TEST  
.... |
```

- Accessing elements of String Python, individual characters of a String can be accessed by using the method of Indexing.
- Indexing allows negative address references to access characters from the back of the String, e.g. -1 refers to the last character, -2 refers to the second last character and so on

[illegible]

Python Program to Access

characters of String

String1 = "Elysium Academy"

print("Initial String: ")

print(String1)

Printing First character

print("\nFirst character of String is: ")

print(String1[0])

Printing Last character

print("\nLast character of String is: ")

print(String1[-1])

RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/Welcome.py

Initial String:
Elysium Academy

First character of String is:
E

Last character of String is:
y



Python input():

- The input() method reads a line from input, converts into a string and returns it.
- Syntax:

`input([prompt])`

Example 1 : How input() works in Python?

`# get input from user`

```
data = input('Enter a String')
```

```
print('The inputted string is:', data)
```

```
RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/  
Enter a String: Am jayaprakash from India  
The inputted string is: : Am jayaprakash from India
```

Example :2

```
print("Enter your name:")  
x = input()  
print("Hello, " + x)
```

```
Enter your name:  
Jayaprakash  
Hello, Jayaprakash
```

Single Input Program

Problem: Get user name and print it

Sample Input

Enter your name: Jaya

Sample Output

Hello Jaya

```
name = input("Enter your name: ")  
print("Hello", name)
```

```
>>>  
= RESTART: C:/Users/JAYAPRAKASH/AppData/Local/Programs/Python/Python38/08.04.2026.  
Enter your name: Jaya  
Hello Jaya  
>>> |
```

String + Integer Input

Problem: Student details

Sample Input

Enter name: Arun

Enter age: 21

Sample Output

Name: Arun

Age: 21

```
name = input("Enter name: ")  
age = int(input("Enter age: "))
```

```
print("Name:", name)  
print("Age:", age)
```

Integer Input Program

Problem Statement : Find square of a number

Sample Input

Enter a number: 5

Sample Output

Square is 25

```
num = int(input("Enter a number: "))  
square = num * num  
print("Square is", square)
```

```
Enter a number: 5  
Square is 25  
>>>
```

Multiple Inputs (Same Line)

Sample Input

Enter two numbers: 10 20

Sample Output

Sum = 30

```
a, b = map(int, input("Enter two numbers: ").split())  
print("Sum =", a + b)
```

```
>>>
```

```
= RESTART: C:/Users/JAYAPRAKASH/AppData/Local/Programs/Python/Python38/08.04.2026.py
```

```
Enter two numbers: 10 20
```

```
Sum = 30
```

```
>>> |
```

Multiple Inputs (Different Lines)

Problem: Total marks

Sample Input

Enter Tamil mark: 80

Enter English mark: 75

Enter Maths mark: 90

Sample Output

Total = 245

```
tamil = int(input("Enter Tamil mark: "))  
english = int(input("Enter English mark: "))  
maths = int(input("Enter Maths mark: "))
```

```
total = tamil + english + maths  
print("Total =", total)
```

```
>>>  
= RESTART: C:/Users/JAYAPRAKASH/AppData/Local/Programs/Python/Python38/08.04.2026.py  
Enter Tamil mark: 80  
Enter English mark: 75  
Enter Maths mark: 90  
Total = 245  
>>> |
```


Real-Time Example (Payment)

Problem statement #1 : Payment details

Sample Input

Enter CreditCard Payment Amount: 1000

Enter UPI Payment Amount: 500

Sample Output

Paid 1000 using Credit Card

Paid 500 using UPI

```
credit = int(input("Enter CreditCard Payment Amount: "))
```

```
upi = int(input("Enter UPI Payment Amount: "))
```

```
print("Paid", credit, "using Credit Card")
```

```
print("Paid", upi, "using UPI")
```

Data Types

Problem Statement #2

Write a Python program to take different types of input (integer, float, string) and display their data types.

Sample Input

Enter integer: 5
Enter float: 3.5
Enter name: John

Sample Output

Integer type: <class 'int'>
Float type: <class 'float'>
String type: <class 'str'>

Program

Data Type Example

```
num = int(input("Enter integer: "))  
decimal = float(input("Enter float: "))  
name = input("Enter name: ")
```

```
print("Integer type:", type(num))  
print("Float type:", type(decimal))  
print("String type:", type(name))
```

Program to enter details of an Student Marks and average and print them. '

```
rollno=int(input("Enter your Roll no\t"))
name=input("Enter your name\t")
math=float(input("Enter your marks in Math\t"))
physics=float(input("Enter your marks in Physics\t"))
chemistry=float(input("Enter your marks in Chemistry\t"))

print("Welcome\t ",name)
print("Your Roll no is\t ",rollno)
print("Marks in Maths is\t ",math)
print("Marks in Physics is\t ",physics)
print("Marks in Chemistry is\t ",chemistry)
print("Average marks is \t",(math+physics+chemistry)/3)
```

>>>

= RESTART: C:\Users\JAYAPRAKASH\AppData\Local\Programs\Python\Python38\ssk program 1.py

Enter your Roll no 1001

Enter your name JayaPrakash S

Enter your marks in Math 87

Enter your marks in Physics 88

Enter your marks in Chemistry 90

Welcome JayaPrakash S

Your Roll no is 1001

Marks in Maths is 87.0

Marks in Physics is 88.0

Marks in Chemistry is 90.0

Average marks is 88.33333333333333

...

Python Operator

Python divides the operators in the following groups:

- Arithmetic operators
- Assignment operators
- Comparison operators
- Logical operators
- Identity operators
- Membership operators

Arithmetic operators

- Arithmetic operators: Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication and division.

OPERATOR	DESCRIPTION	SYNTAX
+	Addition: adds two operands	$x + y$
-	Subtraction: subtracts two operands	$x - y$
*	Multiplication: multiplies two operands	$x * y$
/	Division (float): divides the first operand by the second	x / y
//	Division (floor): divides the first operand by the second	$x // y$
%	Modulus: returns the remainder when first operand is divided by the second	$x \% y$
**	Power : Returns first raised to power second	$x ** y$

Program

```
x = 5
```

```
y = 3
```

```
# Addition of numbers
```

```
print(x + y)
```

```
# Subtraction of numbers
```

```
print(x - y)
```

```
# Multiplication of number
```

```
print(x * y)
```

```
x = 12
```

```
y = 3
```

```
# Divisionof number
```

```
print(x / y)
```



```
X=5
```

```
Y=2
```

```
print(x % y)
```

```
x = 2
```

```
y = 3
```

```
print(x ** y) #same as 2*2*2
```

```
x = 21
```

```
y = 2
```

```
print(x // y)
```

#the floor division // rounds the result down to the nearest whole number

```
RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/
```

```
8  
2  
15  
4.0  
1  
8  
10
```

Relational Operators:

- Relational operators compares the values. It either returns True or False according to the condition

OPERATOR	DESCRIPTION	SYNTAX
>	Greater than: True if left operand is greater than the right	$x > y$
<	Less than: True if left operand is less than the right	$x < y$
==	Equal to: True if both operands are equal	$x == y$
!=	Not equal to - True if operands are not equal	$x != y$
>=	Greater than or equal to: True if left operand is greater than or equal to the right	$x >= y$
<=	Less than or equal to: True if left operand is less than or equal to the right	$x <= y$

```
a = 13
```

```
b = 33
```

```
# a > b is False
```

```
print(a > b)
```

```
# a < b is True
```

```
print(a < b)
```

```
# a == b is False
```

```
print(a == b)
```

```
# a != b is True
```

```
print(a != b)
```

```
# a >= b is False
```

```
print(a >= b)
```

```
# a <= b is True
```

OutPut:

False

True

False

True

False

True

`x = 5`

`y = 3`

`print(x == y)`

`# returns False because 5 is not equal to 3`

`x = 5`

`y = 3`

`print(x != y)`

`# returns True because 5 is not equal to 3`

`x = 2`

`y = 3`

`print(x > y)`

```
x = 3
```

```
y = 3
```

```
print(x >= y)
```

```
x = 5
```

```
y = 3
```

```
print(x <= y)
```

```
# returns False because 5 is neither less than or equal to 3
```

```
RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/
```

```
False
```

```
True
```

```
False
```

```
True
```

```
False
```

Python Assignment Operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
**=	x **= 3	x = x ** 3

```
x = 5
x -= 3
print(x)
```

```
x = 5
x *= 3
print(x)
```

```
x = 5
x /= 3
print(x)
```

```
x = 5
x%=3
print(x)
```

```
x = 3
x **= 3
```

RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/

```
2
15
1.6666666666666667
2
27
```

Python Logical Operators

Logical operators perform Logical AND, Logical OR and Logical NOT operations.

Operator	Description	Example
and	Returns True if both statements are true	<code>x < 5 and x < 10</code>
or	Returns True if one of the statements is true	<code>x < 5 or x < 4</code>
not	Reverse the result, returns False if the result is true	<code>not(x < 5 and x < 10)</code>


```
x = 5
```

```
print(x > 3 and x < 10)
```

```
# returns True because 5 is greater than 3 AND 5 is less than 10
```

```
x = 5
```

```
print(x > 3 or x < 4)
```

```
# returns True because one of the conditions are true (5 is greater than 3, but 5 is not less than 4)
```

```
x = 5
```

```
print(not(x > 3 and x < 10))
```

RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/

True

True

False

Membership operators

in and **not in** are the membership operators; used to test whether a value or variable is in a sequence.

in True if value is found in the sequence

not in True if value is not found in the sequence

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

```
x = ["apple", "banana"]
```

```
print("banana" in x)
```

```
# returns True because a sequence with the value "banana" is in the list
```

```
x = ["apple", "banana"]
```

```
print("pineapple" not in x)
```

```
# returns True because a sequence with the value "pineapple" is not in the list
```

True

True

Python Identity Operators

```
x = ["apple", "banana"]
```

```
y = ["apple", "banana"]
```

```
z = x
```

```
print(x is z)
```

returns True because z is the same object as x

```
print(x is y)
```

returns False because x is not the same object as y, even if they have the same content

```
RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/
True
False
```

```
x = ["apple", "banana"]
```

```
z = x
```

```
print(x is not z)
```

```
# returns False because z is the same object as x
```

Output:

False



Conditional Statements

Python supports the usual logical conditions from mathematics:

Equals: `a == b`

Not Equals: `a != b`

Less than: `a < b`

Less than or equal to: `a <= b`

Greater than: `a > b`

Greater than or equal to: `a >= b`

If statement:

```
a = 33
```

```
b = 200
```

```
if b > a:
```

```
    print("b is greater than a")
```

b is greater than a

elif

elif statement allows you to check multiple expressions for TRUE and execute a block of code as soon as one of the conditions evaluates to TRUE.

Syntax

```
if expression1:  
    statement(s)  
elif expression2:  
    statement(s)  
elif expression3:  
    statement(s)  
else:  
    statement(s)
```

else

```
a = 200
```

```
b = 33
```

```
if b > a:
```

```
    print("b is greater than a")
```

```
else:
```

```
    print("b is not greater than a")
```

Output:

B is not greater than a

else

```
a = 200
```

```
b = 20
```

```
if b > a:
```

```
    print("b is greater than a")
```

```
elif a == b:
```

```
    print("a and b are equal")
```

```
else:
```

```
    print("a is greater than b")
```

Output:

a is greater than b

```
amount= 2000

if amount<1000:
    discount=amount*0.05
    print ("Discount",discount)
elif amount<5000:
    discount=amount*0.10
    print ("Discount",discount)
else:
    discount=amount*0.15
    print ("Discount",discount)
    print ("Net payable:",amount-discount)
```

```
RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/
Discount 200.0
>>> |
```

Nested IF Statements

The syntax of the nested if...elif...else construct may be

If expression1:

 statement(s)

 if expression2:

 statement(s)

 elif expression3:

 statement(s)

 else

 statement(s)

else:

 statement(s)

Python nested IF statements

```
var = 100
if var < 200:
    print ("Expression value is less than 200")
    if var == 150:
        print ("Which is 150")
    elif var == 100:
        print ("Which is 100")
    elif var == 50:
        print ("Which is 50")
    elif var < 50:
        print ("Expression value is less than 50")
else:
    print ("Could not find true expression")

print ("Good bye!")
```

```
>>> ===== RESTART =====
>>>
Expression value is less than 200
Which is 100
Good bye!
>>> |
```

```
var = 185
if var < 200:
    print ("Expression value is less than 200")
    if (var > 150 and var<170):
        print ("Which is greater than 150 less than 170")
    elif (var == 100 or var>180):
        print ("Which is equal to 100 or greater than 180")
    elif var == 50:
        print ("Which is 50")
    elif var < 50:
        print ("Expression value is less than 50")
else:
    print ("Could not find true expression")

print ("Good bye!")
```

```
=== RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37,
Expression value is less than 200
Which is equal to 100 or greater than 180
Good bye!
```



Loops

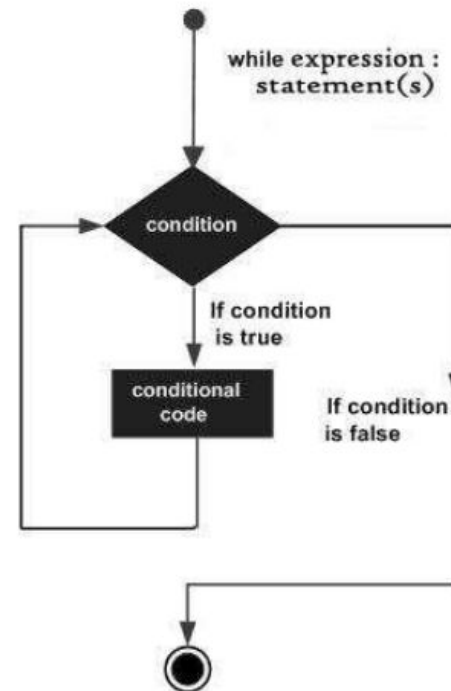
Loop Type	Description
while loop	Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.
for loop	Executes a sequence of statements multiple times and abbreviates the code that manages the loop variable.

while Loop Statements

Syntax:

```
while expression:  
    statement(s)
```

Flow Diagram



example

```
count = 0
while (count < 9):
    print ('The count is:', count)
    count = count + 1
print ("Good bye!")
```

```
i = 1
while i < 6:
    print(i)
    i += 1
```

RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/Welcc

```
The count is: 0
The count is: 1
The count is: 2
The count is: 3
The count is: 4
The count is: 5
The count is: 6
The count is: 7
The count is: 8
Good bye!
```

```
count = 0
```

```
while count < 7:
```

```
    print (count, " is less than 7")
```

```
    count = count + 1
```

```
else:
```

```
    print ("Loop End")
```

```
RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/
```

```
0 is less than 7
```

```
1 is less than 7
```

```
2 is less than 7
```

```
3 is less than 7
```

```
4 is less than 7
```

```
5 is less than 7
```

```
6 is less than 7
```

```
Loop End
```

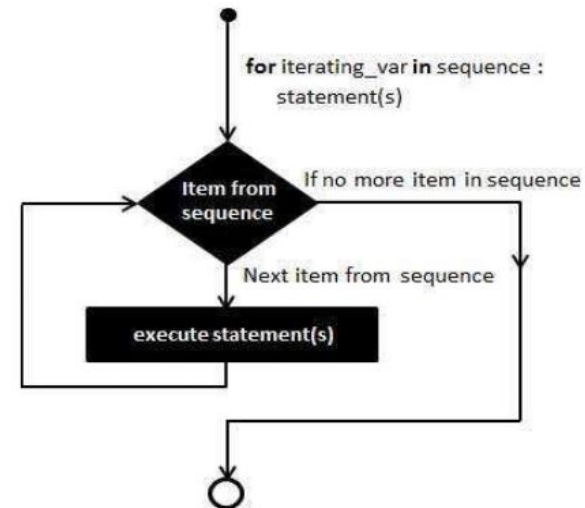
for Loop Statements

The for statement in Python has the ability to iterate over the items of any sequence, such as a list or a string.

Syntax:

```
for iterating_var in sequence:  
    statements(s)
```

Flow Diagram



range() function

The built-in function range() is the right function to iterate over a sequence of numbers. It generates an iterator of arithmetic progressions.

```
>>> range(5)
```

```
range(0, 5)
```

```
>>> list(range(5))
```

```
[0, 1, 2, 3, 4]
```

range() generates an iterator to progress integers starting with 0 upto n-1

```
for var in list(range(5)):
```

```
    print (var)
```

```
RESTART: C:/Users/eapl-15/AppData/Local/Programs/Python/Python37/
```

```
0  
1  
2  
3  
4  
,
```

```
# Python program to
# print all number
# divisible by 3 and 5

# using range to print number
# divisible by 3
for i in range(0, 30, 3):
    print(i)
|

# using range to print number
# divisible by 5
for i in range(0, 50, 5):
    print(i)
```

`range(start, stop, step)`

```
===== RESTART: C:/Python27/fir.py =====
0
3
6
9
12
15
18
21
24
27
0
5
10
15
20
25
30
35
40
45
>>> |
```



Loop Control Statements

Python supports the following control statements.

Control Statement	Description
break statement	Terminates the loop statement and transfers execution to the statement immediately following the loop.
continue statement	Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.
pass statement	The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

Break / Continue statement

1. The break statement is used for Stops the loop immediately

Syntax:

`break`

2. continue Statement

Skips current iteration and moves to next

3. `pass` Statement

a. Does nothing (placeholder)

placeholder for future code

Used when you `plan to write code later`

b. Purpose of pass in Python

pass is a null (empty) statement – it does nothing, but it is used to avoid syntax errors when a statement is required.

1.break

```
for i in range(1, 6):  
    if i == 3:  
        break  
    print(i)
```

Output

1
2

When i = 3, loop stops completely

2. continue Statement

```
for i in range(1, 6):  
    if i == 3:  
        continue  
    print(i)
```

Output

1
2
4
5

When i = 3, it is skipped

Pass statement

To create an empty block

Python requires at least one statement inside blocks (if, for, function, etc.)

```
for i in range(1, 4):
```

```
    pass
```

```
print("Loop completed")
```

```
>>>
```

```
= RESTART: C:\Users\JAYAF
```

```
Loop completed
```

```
>>>
```

```
for i in range(1, 4):
```

```
    print("Loop completed")
```

Empty Condition Block

```
x = 10
```

```
if x > 5:
```

```
    pass
```

```
else:
```

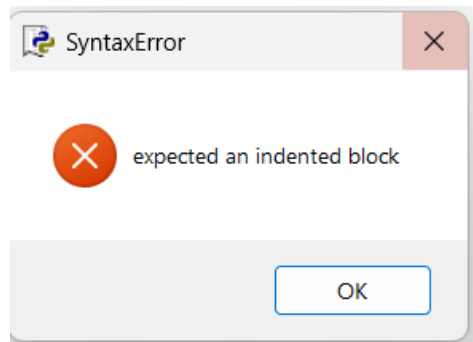
```
    print("Less than 5")
```

```
print("Done")
```

Output

Done

if block is empty but no error occurs



Python format() Method

The format() method is used to insert values into a string and perform string formatting.

During formatting, we use curly braces {} as placeholders. These braces are replaced by the values provided to the format() method.

Basic Syntax

```
"string {}".format(value)
```

Remember:

{ } = Placeholder

format() = Supplies the values

Basic formatting

Simple positional formatting is probably the most common use-case. Use it if the order of your arguments is not likely to change and you only have very few elements you want to concatenate.

Since the elements are not represented by something as descriptive as a name this simple style should only be used to format a relatively small number of elements.

Old

```
'%s %s' % ('one', 'two')
```

New

```
'{} {}'.format('one', 'two')
```

Output

```
o n e   t w o
```

Old

```
'%d %d' % (1, 2)
```

New

```
'{} {}'.format(1, 2)
```

Output

```
1 2
```

Old String format

```
# This prints out "Hello, John!"
```

```
name = "John"
```

```
print("Hello, %s!" % name)
```

```
RESTART: C:\Users\eapl-15\AppData\Local\Programs\Python\Python37\
Hello, John!
```

```
# This prints out "John is 23 years old."
```

```
name = "John"
```

```
age = 23
```

```
print("%s is %d years old." % (name, age))
```

```
RESTART: C:\Users\eapl-15\AppData\Local\Programs\Python\Python37\
John is 23 years old.
```

New String Format {} Method

Basic format()

```
name = "John"
```

```
age = 23
```

```
print("My name is {} and I am {} years old.".format(name, age))
```

,

Output

My name is John and I am 23 years old.

are inserted into the {} placeholders in the same order.

Placeholder	Value
First {}	name → "John"
Second {}	age → 23

How it works

"My name is {} and I am {} years old."

↑ ↑
Placeholder Placeholder



online python jupyter notebook



AI Mode

All

Videos

Images

Forums

Shopping

Short videos

More ▾

Tools ▾



Jupyter Notebook

<https://jupyter.org> > try



Project Jupyter | Try Jupyter

Project Jupyter builds tools, standards, and services for many different use cases. This page has links to interactive demos that allow you to try some of ... [Read more](#)



Jupyter Notebook

<https://jupyter.org>



Jupyter Notebook

The **Jupyter Notebook** is a web-based interactive computing platform. The notebook combines live code, equations, narrative text, visualizations, ...



Jupyter Notebook

<https://jupyter.org> > try-jupyter > lab



Try Jupyter!

WASM powered Jupyter running in the browser. headings in notebooks and supported files. Launcher



/ notebooks /

Name ▲

Modified

📁 16.09.2026

now

📄 cpp-third-party-lib...

7d ago

📄 cpp-tiny-ray-tracer...

7d ago

📄 cpp.ipynb

7d ago

• 📄 Intro.ipynb

7d ago

📄 Lorenz.ipynb

7d ago

📄 Mandelbrot.ipynb

7d ago

📄 PythonProgram.ip...

now

📄 r.ipynb

7d ago

📄 sqlite.ipynb

7d ago

• 📄 Untitled.ipynb

now

📄 untitled1.txt

5m ago

📄 untitled2.py

now

Untitled.ipynb



Code



```
[1]: # This prints out "John is 23 years old."
name = "John"
age = 23
print("%s is %d years old." % (name, age))
```

John is 23 years old.

```
[2]: # This prints out "Hello, John!"
name = "John"
print("Hello, %s!" % name)
```

Hello, John!

[]:

📁 / notebooks / 16.09.2026 /

Name	Modified
• 📄 Untitled.ipynb	2m ago

📄 Untitled.ipynb +

📄 + ✂️ 📄 📄 ▶️ ■️ ↻ ⏏️ Code ▾

```
[1]: # This prints out "John is 23 years old."
name = "John"
age = 23
print("%s is %d years old." % (name, age))
```

John is 23 years old.

```
[2]: # This prints out "Hello, John!"
name = "John"
print("Hello, %s!" % name)
```

Hello, John!

```
[3]: print('jayaprakash')
```

jayaprakash

```
[4]: name = "John"
age = 23

print("My name is {} and I am {} years old.".format(name, age))
```

My name is John and I am 23 years old.

[]:

Python String format() Method {} :

Python 3 format() method is used to perform format operations on string. While formatting string a delimiter {} (braces) is used to replace it with the value.

#named indexes:

```
txt1 = "My name is {fname}, I'm {age}".format(fname = "John", age = 36)
```

#numbered indexes:

```
txt2 = "My name is {0}, I'm {1}".format("John",36)
```

#empty placeholders:

```
txt3 = "My name is {}, I'm {}".format("John",36)
```

```
print(txt1)
```

```
print(txt2)
```

```
print(txt3)
```

```
My name is John, I'm 36
My name is John, I'm 36
My name is John, I'm 36
```

Python format() function example

Variable declaration

str = "Java"

str2 = "C#"

Calling function

str3 = "{} and {} both are programming languages".format(str,str2)

Displaying result

print(str3)

Java and C# both are programming languages

Python program to do arithmetical operations

Store input numbers:

```
num1 = input('Enter first number: ')
```

```
num2 = input('Enter second number: ')
```

Add two numbers

```
sum = float(num1) + float(num2)
```

Subtract two numbers

```
min = float(num1) - float(num2)
```

Multiply two numbers

```
mul = float(num1) * float(num2)
```

#Divide two numbers

```
div = float(num1) / float(num2)
```

Display the sum

```
print('The sum of {0} and {1} is {2}'.format(num1, num2, sum))
```

Display the subtraction

```
print('The subtraction of {0} and {1} is {2}'.format(num1, num2, min))
```

Display the multiplication

```
print('The multiplication of {0} and {1} is {2}'.format(num1, num2, mul))
```

Display the division

```
print('The division of {0} and {1} is {2}'.format(num1, num2, div))
```

```
Enter first number: 10
Enter second number: 2
The sum of 10 and 2 is 12.0
The subtraction of 10 and 2 is 8.0
The multiplication of 10 and 2 is 20.0
The division of 10 and 2 is 5.0
```

Python program to swap two variables

```
# Python swap program
```

```
x = input('Enter value of x: ')
```

```
y = input('Enter value of y: ')
```

```
# create a temporary variable and swap the values
```

```
temp = x
```

```
x = y
```

```
y = temp
```

```
print('The value of x after swapping: {}'.format(x))
```

```
print('The value of y after swapping: {}'.format(y))
```

The value of x after swapping: 10
The value of y after swapping: 5

Python Program to Check if a Number is Odd or Even

```
num = int(input("Enter a number: "))  
if (num % 2) == 0:  
    print("{0} is Even number".format(num))  
else:  
    print("{0} is Odd number".format(num))
```

```
Enter a number: 2  
2 is Even number  
>>>  
=====
```

Enter a number: 3
3 is Odd number

Python Program to check if a Number is Positive, Negative or Zero

```
num = float(input("Enter a number: "))
```

```
if num > 0:
```

```
    print("{0} is a positive number".format(num))
```

```
elif num == 0:
```

```
    print("{0} is zero".format(num))
```

```
else:
```

```
    print("{0} is negative number".format(num))
```

```
Enter a number: 10
10.0 is a positive number
>>>
=====
Enter a number: 0
0.0 is zero
>>>
=====
Enter a number: -2
-2.0 is negative number
```

